

Assignment 1 (100 Marks)

COMP2123 Data Structures and Algorithms
Semester 2, 2026

This assignment is due in **Week 5** and must be submitted through Gradescope.

Work in **groups of three**. One group member must upload the PDF and add the other two group members to the Gradescope submission.

Submit a PDF containing typed answers. Handwritten answers and PDFs consisting of scanned images of text will not be accepted. The PDF must contain selectable text so that it can be read by Turnitin.

Type the student IDs of all three group members at the top left of the first page. Do not type any names. Do not include a cover page. Submit only the answers and do not copy the problem statements into the submission.

The maximum submission length is **five pages**. If a submission contains more than five pages, only the first five pages will be reviewed.

For every problem, begin with a clear description of the main idea. Give enough detail for the algorithm to be implemented, prove that the algorithm is correct, and analyse its worst case running time and additional space usage. Pseudocode is optional. Pseudocode without an explanation in plain English is not sufficient.

The submitted work must be produced by the registered group. Any source outside the lecture, tutorial, and recommended textbook material must be cited.

Problem 1.

[20 points]

Nested cryptographic transcript

A cryptographic protocol may execute several proof components inside one another. An audit system records the beginning and end of each component in a transcript

$$E[0], E[1], \dots, E[n-1].$$

Each entry has one of the following forms:

$$\text{open}(k),$$

meaning that the proof component with identifier k begins, or

$$\text{close}(k),$$

meaning that the proof component with identifier k ends.

Every identifier appears in at most one open event and at most one close event.

A transcript is *valid* when both of the following conditions hold:

1. Every $\text{close}(k)$ event matches the most recently opened component that has not already been closed.
2. Every opened component is closed before the transcript ends.

The number of components that are open at the same time is called the current nesting depth. The largest nesting depth reached anywhere in the transcript is called the *maximum nesting depth*.

The algorithm must return one of the following outputs:

1.

`(valid, d)`,

where d is the maximum nesting depth, if the transcript is valid.

2.

`(invalid, i)`,

where i is the index of the first close event that cannot be matched correctly. Transcript indices start at 0.

3.

`incomplete`,

if every close event is matched correctly, but one or more components remain open when the transcript ends.

The transcript may be empty. The empty transcript is valid and has maximum nesting depth 0.

Examples. For the transcript

$$E = [\text{open}(7), \text{open}(3), \text{close}(3), \\ \text{open}(9), \text{close}(9), \text{close}(7)],$$

the correct output is

`(valid, 2)`.

For the transcript

$$E = [\text{open}(7), \text{open}(3), \text{close}(7), \text{close}(3)],$$

the correct output is

`(invalid, 2)`,

because component 3 must be closed before component 7.

For the transcript

$$E = [\text{open}(7), \text{open}(3), \text{close}(3)],$$

the correct output is

`incomplete`,

because component 7 remains open.

Design an algorithm that produces the required output in $O(n)$ worst case time.

- Describe the algorithm in plain English.
- Prove that the algorithm returns the correct output.
- Analyse the worst case running time and additional space used by the algorithm.

Problem 2.

[30 points]

Product recall in a catalogue tree

A retailer organises its products using a rooted catalogue tree T containing n nodes. Every node has a distinct label.

Each leaf represents one product. Every leaf is marked either *safe* or *recalled*. Internal nodes represent product categories and do not have a mark. At least two leaves are recalled.

Issuing a recall at a node v removes every product represented by a leaf in the subtree rooted at v .

A node v is a *valid recall node* if the subtree rooted at v contains every recalled product in the entire catalogue.

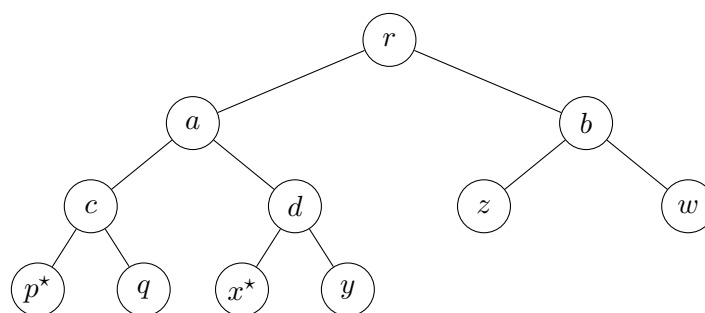
The depth of a node is the number of edges on the path from the root to that node. Among all valid recall nodes, the retailer chooses the node of maximum depth. This keeps the recall within the smallest possible part of the catalogue.

The algorithm must return an ordered pair (v, s) , where

- v is the valid recall node of maximum depth
- s is the number of safe product leaves in the subtree rooted at v

The value s may be 0.

Example. In the following tree, a star marks a recalled product.



The recalled products are p and x . The subtree rooted at a contains both recalled products. Neither the subtree rooted at c nor the subtree rooted at d contains both recalled products. Therefore, a is the valid recall node of maximum depth.

The safe products in the subtree rooted at a are q and y . The correct output is therefore

$(a, 2)$.

The tree is not assumed to be binary. The children of a node may be examined using the Tree ADT operations from the lecture. The label of any node and the mark of any leaf can be inspected in $O(1)$ time.

Design an algorithm that produces the required output in $O(n)$ worst case time.

- (a) Describe the algorithm in plain English.
- (b) Prove correctness, that the returned node is the valid recall node of maximum depth. Also prove that the returned number of safe products is correct.
- (c) Analyse the worst case running time and additional space used by the algorithm.

Problem 3.

[50 points]

Risk threshold audit in a certificate registry

A certificate registry stores n active certificates in a binary search tree T . Each certificate has

1. a distinct integer serial number s
2. a positive integer risk score $r(s)$

The binary search tree is ordered by serial number. Let h be the height of T when the tree is nonempty. The tree is not assumed to be balanced.

A security auditor chooses an integer lower bound a and a positive integer risk threshold B . The audit begins with the first stored certificate whose serial number is at least a , and then examines certificates in increasing serial number order.

Define

$$\text{auditWindow}(a, B)$$

to return the ordered pair (x, m) , where x is the smallest serial number stored in T such that $x \geq a$ and

$$\sum_{\substack{s \text{ is stored in } T \\ a \leq s \leq x}} r(s) \geq B,$$

and where

$$m = \max_{\substack{s \text{ is stored in } T \\ a \leq s \leq x}} r(s).$$

Thus, x is the first serial number at which the accumulated risk from serial number a reaches the threshold, and m is the largest individual risk score encountered in that audit window.

If no such serial number x exists, the operation returns **None**. In particular, an empty tree returns **None**.

Example. Suppose the registry contains the following pairs, written as

$$(\text{serial number}, \text{risk score}) : (2, 4), \quad (5, 1), \quad (8, 6), \quad (11, 3), \quad (14, 5).$$

Starting from serial number 4, the examined risk scores are 1, 6, 3, 5. The threshold 7 is first reached at serial number 8, and the largest risk score encountered is 6. Therefore,

$$\text{auditWindow}(4, 7) = (8, 6).$$

Starting from serial number 6, the threshold 7 is first reached after examining serial numbers 8 and 11. Therefore,

$$\text{auditWindow}(6, 7) = (11, 6).$$

Some other correct outputs are

$$\text{auditWindow}(1, 5) = (5, 4),$$

$$\text{auditWindow}(9, 6) = (14, 5),$$

and

$$\text{auditWindow}(12, 6) = \text{None}.$$

If certificate 8 is removed, then

$$\text{auditWindow}(4, 7) = (14, 5).$$

If a new certificate with serial number 7 and risk score 8 is then inserted, the correct output becomes

$$\text{auditWindow}(4, 7) = (7, 8).$$

Assume that the standard binary search tree implementations of $\text{put}(s, r)$ and $\text{remove}(s)$ from the lecture are already available. Every call to $\text{put}(s, r)$ inserts a serial number that is not currently stored in the tree. Every call to $\text{remove}(s)$ removes a serial number that is currently stored in the tree. When the standard removal algorithm replaces a node entry by its inorder successor entry, both the serial number and the risk score are copied.

A constant number of additional integer fields may be stored at each node that contains a certificate. The standard pointer operations for insertion and removal do not need to be reproduced. It is sufficient to explain how the additional information is maintained in every case of the standard insertion and removal algorithms.

All risk scores, sums of risk scores, and other stored integer values fit in fixed size integers. Each arithmetic operation and comparison takes $O(1)$ time.

Requirements.

1. If the tree is nonempty, $\text{auditWindow}(a, B)$ must run in $O(h)$ worst case time. If the tree is empty, it must run in $O(1)$ time.
2. put and remove must continue to run in $O(h)$ worst case time.

3. The total space used by the binary search tree must be $O(n)$.
- (a) Give an algorithm for `auditWindow( $a, B$ )`.
- (b) Prove that `auditWindow( $a, B$ )` returns the required pair (x, m) , or returns **None** exactly when no valid serial number x exists.
- (c) Analyse the worst case running time of `auditWindow( $a, B$ )`.

Sample Solutions

Problem 1:

- (a) Scan E from left to right, maintaining a stack S of identifiers and a counter d initialised to 0.

On `open( $k$ )`, push k and set $d = \max(d, |S|)$.

On `close( $k$ )` at index i , return `(invalid,  $i$ )` if S is empty or $\text{top}(S) \neq k$, and otherwise pop.

If the scan completes, return `incomplete` when S is nonempty and `(valid,  $d$ )` otherwise. The empty transcript leaves S empty and $d = 0$, giving `(valid, 0)`.

- (b) *Invariant.* After any prefix processed without returning, S holds exactly the components opened in that prefix and not yet closed, ordered by opening time with the most recent on top. This is true for the empty prefix. An `open( $k$ )` makes k the most recently opened component and pushes it on top. A successful `close( $k$ )` removes exactly the most recent one.

Invalid. By the invariant, at index i the most recently opened component not yet closed is $\text{top}(S)$ when S is nonempty, and no such component exists when S is empty. So `close( $k$ )` satisfies condition 1 if and only if S is nonempty and $\text{top}(S) = k$, which is the test performed. Every earlier close event passed this test, so the index reported is the first failure.

Valid and incomplete. If the scan completes, condition 1 holds at every close event. The invariant then says S holds exactly the components still open, so condition 2 holds if and only if S is empty. This is the distinction the algorithm makes.

Depth. By the invariant $|S|$ is the current nesting depth, and $|S|$ increases only at a push. Hence the maximum nesting depth is the largest value of $|S|$ immediately after a push, which is d .

- (c) One pass with $O(1)$ stack work per event gives $O(n)$ worst case time. The stack holds at most one identifier per open event, so the additional space is $O(n)$, attained when every event is an open event.

Problem 2:

- (a) Traverse the tree once in postorder to obtain R , the number of recalled leaves in the whole catalogue.

Traverse it again in postorder. The traversal returns for each node v the pair $(R(v), S(v))$, the numbers of recalled and of safe leaves in the subtree rooted at v . A leaf returns $(1, 0)$ if it is recalled and $(0, 1)$ if it is safe. An internal node returns the componentwise sum of the pairs

returned by its children. Record the first node v whose returned pair has $R(v) = R$, together with its $S(v)$, and ignore every later such node. Return that pair. No depth needs to be tracked.

(b) *Counts.* By induction on the height of the subtree: a leaf reads its pair from its mark, and for an internal node the leaf sets of the subtrees of its children partition the leaf set of its own subtree, so componentwise addition is correct.

Valid nodes. Every v has $R(v) \leq R$, with equality if and only if every recalled leaf lies in the subtree of v . So the nodes with $R(v) = R$ are exactly the valid recall nodes.

The valid nodes form a chain. If u and v are valid and neither is an ancestor of the other, their subtrees are disjoint, so $2R = R(u) + R(v) \leq R$ and hence $R = 0$, contradicting the assumption that at least two leaves are recalled. Therefore all valid nodes lie on one path from the root, a valid node has maximum depth if and only if it has no valid proper descendant, and that node is unique.

Selection. A postorder traversal returns the pair of every proper descendant of v before that of v , so the first valid node returned has no valid proper descendant and is the node of maximum depth. The value recorded with it is $S(v)$, the number of safe product leaves in its subtree.

Remark. One traversal also suffices. Record v whenever at least two children of v return a positive recalled count, overwriting any earlier record. Every ancestor of the deepest valid node has exactly one child with a positive count, so every recorded node is that node or one of its descendants, and postorder records it last.

(c) Each traversal visits every node once and does $O(1)$ work per parent and child pair. The tree has $n - 1$ such pairs, so the total is $O(n)$ worst case time. The recursion holds one frame per level, each keeping a running pair, so the additional space is $O(h)$, which is $O(n)$ in the worst case because the tree need not be binary. Storing the pair at every node instead also uses $O(n)$.

Problem 3:

(a) **Augmentation.** Write T_v for the subtree rooted at v . Each node v stores two integers,

$$\text{sub}(v) = \sum_{u \in T_v} r(u), \quad \text{mx}(v) = \max_{u \in T_v} r(u),$$

both defined to be 0 for an empty subtree, which is safe because every risk score is positive. They satisfy

$$\text{sub}(v) = \text{sub}(v.\text{left}) + r(v) + \text{sub}(v.\text{right}), \quad \text{mx}(v) = \max\{\text{mx}(v.\text{left}), r(v), \text{mx}(v.\text{right})\}.$$

Maintenance. Run the standard insertion or removal. Only the ancestors of the position that changed structurally have a different subtree, so recompute the two fields with the formulas above at every node on the path from that position up to the root, working upwards. Each recomputation reads the fields of the children, which are already correct, and costs $O(1)$. When the removed key sits at a node z with two children, the successor entry is copied into z and the successor node is removed from the right subtree of z . That node is a descendant of z , so the upward recomputation passes through z and repairs it as well.

Notation. For a stored key y let $\text{pre}(y) = \sum_{s \leq y} r(s)$, summed over stored keys. Because risk scores are positive, pre is strictly increasing along the stored keys.

Step 1. Descend from the root computing $P = \sum_{s < a} r(s)$ over stored keys. At v , if $v.\text{key} < a$ add $\text{sub}(v.\text{left}) + r(v)$ to P and go right, otherwise go left.

Step 2. Let $C = P + B$. Return **None** if the tree is empty or $C > \text{sub}(\text{root})$. Otherwise descend from the root, writing $L = \text{sub}(v.\text{left})$. Go left if $C \leq L$. Stop at v if $L < C \leq L + r(v)$. Otherwise replace C by $C - L - r(v)$ and go right. Let x be the key where the descent stops.

Step 3. Descend from the root, going right while the key is below a and left while it is above x , until reaching the node w with $a \leq w.\text{key} \leq x$. Set $m = r(w)$.

From $w.\text{left}$, walk down: at u , if $u.\text{key} \geq a$ set $m = \max\{m, r(u), \text{mx}(u.\text{right})\}$ and go left, otherwise go right.

From $w.\text{right}$, walk down: at u , if $u.\text{key} \leq x$ set $m = \max\{m, r(u), \text{mx}(u.\text{left})\}$ and go right, otherwise go left.

Return (x, m) .

(b) *Fields.* The two formulas are the definitions split over the root of the subtree, its left subtree and its right subtree, so induction on the height of the subtree gives correctness. An update changes T_v only for ancestors v of the changed position, and those nodes are recomputed from correct children, so the invariant is restored.

Step 1. If $v.\text{key} < a$ then v and every node of $v.\text{left}$ have keys below a , and every stored key below a in T_v that is not yet counted lies in $v.\text{right}$. If $v.\text{key} \geq a$ then v and every node of $v.\text{right}$ have keys at least a . So the descent adds every stored key below a exactly once and adds nothing else, giving P .

Step 2 (target). For a stored $y \geq a$ the requirement $\sum_{a \leq s \leq y} r(s) \geq B$ is exactly $\text{pre}(y) \geq P + B = C$. Every stored $y < a$ has $\text{pre}(y) \leq P < C$, because B is positive, so no key below a can qualify and the constraint $y \geq a$ is automatic. The task is therefore to return the smallest stored y with $\text{pre}(y) \geq C$. Since pre is increasing, such a y exists if and only if $\text{sub}(\text{root}) = \text{pre}(y_{\max}) \geq C$, where $y_{\max}$ is the largest stored key. That is exactly the test for **None**, so **None** is returned precisely when no valid x exists.

Step 2 (descent). The invariant is that the answer is the smallest stored key in T_v whose prefix sum within T_v reaches the current C , and that $1 \leq C \leq \text{sub}(v)$. Every key of $v.\text{left}$ precedes $v.\text{key}$, which precedes every key of $v.\text{right}$. So if $C \leq L$ the answer lies in $v.\text{left}$ with C unchanged, if $L < C \leq L + r(v)$ the answer is $v.\text{key}$, and otherwise it lies in $v.\text{right}$ with C reduced by the risk of everything at or before v . The invariant is preserved in each case, so the descent returns the required x .

Step 3 (node w exists). The key x is stored and $a \leq x$, so at least one stored key lies in $[a, x]$ and the descent cannot reach an empty subtree.

Step 3 (coverage). Every node passed above w was left by going right because its key is below a , or by going left because its key is above x , so no stored key of $[a, x]$ lies outside T_w . Within T_w , consider the left walk. Every visited u has $u.\text{key} < w.\text{key} \leq x$. If $u.\text{key} \geq a$ then u lies in $[a, x]$ and every key of $u.\text{right}$ lies strictly between $u.\text{key}$ and $w.\text{key}$, hence also in $[a, x]$, so both may be included, and the only remaining candidates are in $u.\text{left}$. If $u.\text{key} < a$ then u and all of $u.\text{left}$ are below a , so nothing is included and the walk continues into $u.\text{right}$. The right walk is symmetric with the roles of a and x exchanged. Together with w this covers every stored key of $[a, x]$ and no other, so m is the maximum risk score in the window.

(c) Step 1 and Step 2 each follow one downward path. Step 3 follows one downward path to w and then two more. Each path visits at most $h + 1$ nodes and each node costs $O(1)$, so `auditWindow( $a, B$ )` runs in $O(h)$ worst case time. On an empty tree Step 1 stops immediately and Step 2 returns `None`, which is $O(1)$.

Insertion and removal recompute two fields at $O(1)$ cost on a path of at most $h + 1$ nodes, so both remain $O(h)$. Each node carries two extra integers, so the total space is $O(n)$.